Java Interview Questions - Set 1

1. What are static blocks and static initializers in Java?

A static block (also called a static initializer) is a block of code that runs once when the class is first loaded by the JVM. It's used to initialize static fields with logic more complex than a single expression.

Syntax:

```
public class Database {
    private static final Map<String, String> CONFIG;

    static {
        CONFIG = new HashMap<>();
        CONFIG.put("host", System.getenv("DB_HOST"));
        CONFIG.put("port", System.getenv("DB_PORT"));
        // logic that needs more than one line
    }
}
```

Key points:

- Runs once, when the class is loaded.
- Runs before any instance is created or any static method is called.
- A class can have multiple static blocks. They run in source order.
- Can't throw checked exceptions. If one throws an unchecked exception, class loading fails with `ExceptionInInitializerError`.

When to use:

- Loading config from environment variables.
- Building a lookup map.
- Registering with a framework.
- Anything where a static field needs setup beyond `= someValue`.

Keep them short. Heavy work in static blocks slows down class loading.

2. How do you call one constructor from another constructor?

Two ways, depending on whether you're calling another constructor in the same class or in the parent class.

Same class: `this(...)`

```java
public class User {
    private String name;
    private int age;
    private boolean active;

    public User(String name) {
        this(name, 0, true); // call the three-arg constructor
    }

    public User(String name, int age) {
        this(name, age, true);
    }

    public User(String name, int age, boolean active) {
        this.name = name;
        this.age = age;
        this.active = active;
    }
}
```

Parent class: `super(...)`

```java
public class Manager extends User {
    private String department;

    public Manager(String name, int age, String department) {
        super(name, age); // call the parent's two-arg constructor
        this.department = department;
    }
}
```

Rules:

- `this(...)` or `super(...)` must be the first statement in the constructor.
- You can use one or the other, never both in the same constructor.
- If you don't write `super(...)`, the compiler inserts a call to the no-arg parent constructor. If the parent has no no-arg constructor, you must call `super(...)` explicitly.
- Constructor chaining helps avoid duplicating initialization logic.

Recent Java versions added "flexible constructor bodies", letting you run validation logic before calling `super` or `this`. For older versions (the common case), the strict "first statement" rule applies.

3. What is method overriding in Java?

Overriding is when a subclass provides its own implementation of a method that's already defined in its parent class. At runtime, Java picks the subclass version based on the actual object type.

```java
public class Animal {
    public void sound() {
        System.out.println("some sound");
    }
}

public class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("bark");
    }
}

Animal a = new Dog();
a.sound(); // prints "bark" via runtime dispatch
```

Rules for a valid override:

- Same method name and parameter list as the parent.
- Same or covariant return type (subclass can return a subtype).
- Same or weaker access modifier (`protected` can become `public`, but not `private`).
- Can't declare broader checked exceptions than the parent.
- Use the `@Override` annotation. The compiler catches typos that would otherwise silently create a new method.

You can override:

- `public`, `protected`, and package-private methods of any parent class.
- Methods declared in interfaces (including default methods).

You can't override:

- `private` methods (they're not visible to subclasses).
- `static` methods (they're hidden, which is a different concept).
- `final` methods (the `final` keyword blocks overriding).

Overriding is the foundation of polymorphism in Java.

4. What is the super keyword in Java?

`super` refers to the immediate parent class. It has three main uses.

Call the parent constructor:

```java
public class Dog extends Animal {
    public Dog(String name) {
        super(name); // call Animal's constructor
    }
}
```

Must be the first statement in the constructor.

Call an overridden parent method:

```
public class Dog extends Animal {
    @Override
    public void sound() {
        super.sound(); // call Animal.sound() first
        System.out.println("then bark");
    }
}
```

Useful when the subclass wants to extend the parent's behavior rather than replace it.

Access a parent field hidden by a subclass field:

```
public class Animal {
    protected String name = "animal";
}

public class Dog extends Animal {
    protected String name = "dog";

    public void show() {
        System.out.println(name);       // "dog"
        System.out.println(super.name); // "animal"
    }
}
```

Hiding fields like this is rarely a good idea. Mention it in interviews. Avoid it in real code.

`super` only goes one level up. It doesn't chain through multiple ancestors.

5. Difference between method overloading and method overriding

Both involve methods with the same name, but they solve different problems.

Aspect	Overloading	Overriding
Where	Same class	Subclass redefines a parent method
Method signature	Different parameter list	Same parameter list
Return type	Can differ	Same or covariant
Access modifier	Any	Same or wider
Resolution	Compile time (static dispatch)	Runtime (dynamic dispatch)
Purpose	Different ways to call a method	Polymorphic behavior across types
Inheritance required?	No	Yes
@Override	Doesn't apply	Recommended

Overloading example:

```
public class Printer {
    public void print(String s) {
        System.out.println(s);
    }

    public void print(int i) {
        System.out.println(i);
    }

    public void print(String s, int times) {
        for (int i = 0; i < times; i++) {
            System.out.println(s);
        }
    }
}
```

Same name `print`, different parameter lists. The compiler picks which version to call based on the arguments.

Overriding example:

```java
public class Animal {
    public void sound() {
        System.out.println("some sound");
    }
}

public class Dog extends Animal {
    @Override
    public void sound() {
        System.out.println("bark");
    }
}
```

Same signature in parent and subclass. The JVM picks which version to run based on the object's actual type at runtime.

6. Difference between abstract class and interface

Covered in detail in the dedicated cheat sheet `java-abstract-interface.pdf`. Skipped here to avoid duplication.

7. Why is Java platform independent?

Two reasons.

1. Java compiles to bytecode. When you run `javac`, the output is a `.class` file containing platform-neutral bytecode rather than CPU-specific machine instructions.
2. The JVM is platform-specific. Each operating system and CPU has its own JVM implementation. The JVM reads bytecode and translates it into the right native instructions at runtime.

So one compiled JAR file runs on Windows, Linux, Mac, or any platform that has a JVM. The slogan "write once, run anywhere" (WORA) captures the idea.

```
Source code (.java)
        |
        v
   javac compiler
        |
        v
   Bytecode (.class)
        |
   +----------+----------+
   v          v          v
Windows JVM  Linux JVM  Mac JVM
```

Trade-off: the JVM adds startup cost and runtime overhead compared to native code. Tools like GraalVM (ahead-of-time compilation) reduce this for use cases where startup time matters.

Note: Java the language is platform-independent. The JVM itself isn't. Each platform needs its own JVM build.

8. What is method overloading in Java?

Method overloading lets you define multiple methods with the same name but different parameter lists in the same class. The compiler picks the right one based on the arguments at the call site.

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int a, int b, int c) {
        return a + b + c;
    }
}

Calculator c = new Calculator();
c.add(1, 2);        // int version
c.add(1.5, 2.5);    // double version
c.add(1, 2, 3);     // three-int version
```

What counts as a "different" parameter list:

- Different number of parameters.
- Different types of parameters.
- Different order of types (e.g., `(int, String)` vs `(String, int)`).

What doesn't count:

- Different return type alone.
- Different parameter names.
- Different access modifier or `throws` clause.

This is compile-time polymorphism. The compiler resolves the call when it parses the source. Compare to overriding, which is runtime polymorphism.

9. What is the difference between C++ and Java?

The two languages look similar on the surface but make different trade-offs.

Aspect	C++	Java
Memory management	Manual (`new` / `delete`)	Automatic (garbage collector)
Pointers	Yes, with arithmetic	References only, no arithmetic
Multiple inheritance	Yes, for classes	Only for interfaces
Compilation	To native code	To bytecode (runs on JVM)
Platform independence	Recompile per platform	Same bytecode runs anywhere
Operator overloading	Yes	No (except `+` for `String`)
Header files	Yes (`.h` and `.cpp`)	No, just `.java`
Preprocessor (`#include` , `#define`)	Yes	No
`struct` and `union`	Yes	No
`goto`	Yes	Reserved keyword but unusable
Standard library	STL	Java SE (Collections, `java.time` , etc.)
Use cases	Systems, games, performance	Web backends, Android, enterprise

Java's design simplified C++ in deliberate ways:

- No pointer arithmetic, because pointer bugs cause crashes and security holes.
- No multiple class inheritance, to avoid the diamond problem.
- No manual memory management, because GC eliminates entire categories of bugs.
- No preprocessor, because macros make code hard to read.

The cost: Java pays a runtime overhead for the JVM and GC. C++ stays closer to the metal. Pick C++ for systems programming, embedded work, or workloads where every cycle matters. Pick Java for almost everything else.

10. What is the JIT compiler?

JIT stands for Just-In-Time compiler. It's part of the JVM that compiles bytecode into native machine code while the program runs.

How it works:

1. The JVM starts by interpreting bytecode line by line. Fast to start, slow per instruction.

2. The JVM profiles which methods get called often. These are the “hot” methods.
3. The JIT compiler translates hot methods into native machine code tuned for the current CPU.
4. Future calls to those methods skip the interpreter and run as native code.

That’s why Java code often gets faster after running for a while. The JIT has had time to find and recompile the hot paths.

HotSpot (the default JVM) ships two JIT compilers that work together (tiered compilation):

- C1 (client compiler): compiles quickly with light optimization. Used for warmup.
- C2 (server compiler): compiles slowly with aggressive optimization. Used for code that stays hot.

Common JIT optimizations:

- Inlining: replace a method call with the method’s body.
- Dead code elimination: remove code that doesn’t affect output.
- Loop unrolling: expand small loops to skip the branch overhead.
- Escape analysis: stack-allocate objects that don’t outlive the method.

JIT vs AOT (Ahead-Of-Time):

- JIT: compiles at runtime. Slower startup, faster steady-state.
- AOT (GraalVM Native Image, etc.): compiles at build time. Faster startup, no warmup needed. Common in serverless and CLI tools.

Quick reference

Question	One-line answer
Static block	Code that runs once when the class loads, used for static field setup
`this(...)`	Calls another constructor in the same class
`super(...)`	Calls the parent class constructor
Method overriding	Subclass redefines a parent method, resolved at runtime
`super` keyword	Reference to the immediate parent (constructor, method, or field)
Overloading vs overriding	Same class + different params vs subclass + same signature
Platform independence	Bytecode runs on any JVM
Method overloading	Same name, different parameter list, resolved at compile time
C++ vs Java	Java has GC, no pointers, single inheritance, runs on JVM
JIT compiler	Compiles bytecode to native code at runtime. HotSpot uses tiered C1 + C2